

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 01 | Python Foundations for DSA & Your Developer Toolkit

Week 1 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 01 at a glance

Item	Details
Lab title	Python Foundations for DSA & Your Developer Toolkit
Week	1 of 16
Prerequisites	None. This lab starts from zero.
Tools	Google Colab (or Jupyter / VS Code), GitHub account, LeetCode account
Files	Lab01_Instructor.ipynb , Lab01_Student.ipynb
Next lab	Lab 02: Complexity Analysis (Big-O)

1. Learning Objectives

By the end of this lab, you will be able to:

1. Set up a complete developer toolkit: **Google Colab, GitHub and LeetCode**.
2. Explain in simple words what a **data structure** and an **algorithm** are, with real-life examples.
3. List and check the **five properties** of a correct algorithm.
4. Write Python programs using **variables, data types, input/output, conditions, loops and functions**.
5. Use Python's built-in collections: **list, tuple, dictionary and set**.
6. Understand how Python stores data in memory (**references, mutable vs immutable**), which is essential for linked lists and trees later.
7. Solve a problem using the professional workflow: **Understand** → **Pseudocode** → **Dry Run** → **Code** → **Test**.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-1	Explain basic data structures and algorithm properties	Understand (C2)
CLO-2	Implement solutions to simple problems in a programming language	Apply (C3)
CLO-4	Use professional tools (version control, online judges)	Apply (P3)

2. Motivation: Why Should You Learn DSA?

Imagine you open Daraz and type “wireless earbuds”. Within a fraction of a second, you see the best results out of **millions of products**. How? Not because the computer is magic, but because engineers chose the **right way to store the data** (data structure) and the **right steps to search it** (algorithm). That is exactly what this course teaches you.

2.1 DSA in Your Daily Life

App you use	What happens behind the scenes	DSA used
Google Maps / Careem / inDrive	Finds the shortest route from your home to the university	Graphs, Dijkstra's algorithm, priority queue
WhatsApp	Shows recent chats on top; delivers messages in order	Queues, linked lists, hashing

App you use	What happens behind the scenes	DSA used
Daraz / Amazon search	Finds and ranks products instantly	Hash tables, trees, sorting
Ctrl + Z (Undo) in MS Word	Remembers your last actions in reverse order	Stack
Browser Back / Forward	Moves between visited pages	Doubly linked list / two stacks
Spotify / YouTube playlists	Next and previous song	Linked list
NADRA / bank records	Finds your record among crores of people using CNIC	Hashing, B-trees (databases)



2.2 DSA in Industry and Jobs

- Technical interviews at Google, Microsoft, Amazon and Meta, and at Pakistani software companies such as **Systems Limited, Arbisoft, 10Pearls, NetSol and Folio3**, are largely based on DSA problems.
- Platforms like **LeetCode, HackerRank and Codeforces** are used by companies to shortlist candidates. You will start using LeetCode from today.
- Freelancers and startup developers who understand DSA write software that stays fast when users grow from 100 to 1,000,000.



2.3 DSA in Research

- **Artificial Intelligence:** neural networks are built on arrays and matrices (tensors); search algorithms power game-playing AI and route planning in robotics.
- **Vector databases** used by ChatGPT-style systems rely on special graphs (e.g., HNSW) to search millions of embeddings quickly.
- **Bioinformatics:** DNA sequences are searched using string algorithms and hashing.
- **Remote Sensing & GIS:** satellite images are 2D/3D arrays; spatial data uses trees such as quadtrees and R-trees.

2.4 Why DSA Matters Even More in 2026–2030

The AI era does not replace DSA, it raises the bar

AI coding assistants can now write code in seconds. So what will make **you** valuable? The ability to **judge** whether that code is correct, efficient and scalable. Only someone who understands DSA can tell when AI-generated code will crash with 10 million users or waste server money.

- Companies will hire people who can **design solutions**, not just type code.
- Big data, AI and cloud systems all run on efficient data structures.
- DSA trains your **problem-solving mind**, a skill that no tool can replace.

3. Part A: Setting Up Your Developer Toolkit

Professional developers do not just write code; they **save, share and test** it. In this part you will create three accounts that you will use for the whole semester and beyond.

3.1 Google Colab: Run Python Without Installing Anything

1. Open **colab.research.google.com** and sign in with your Gmail account.
2. Click **File** → **Upload notebook** and select **Lab01_Student.ipynb** (provided by your instructor).
3. A notebook has two kinds of cells: **Text cells** (explanations) and **Code cells** (Python code).
4. Click inside a code cell and press **Shift + Enter** to run it. The output appears below the cell.
5. Save a copy to your Google Drive: **File** → **Save a copy in Drive**.

Note

You may also use **Jupyter Notebook** (via Anaconda) or **VS Code** with the Python and Jupyter extensions installed on your laptop. All lab notebooks work in all three.

3.2 GitHub: Your Professional Portfolio

GitHub is where developers store their code online. Recruiters often check a candidate's GitHub profile before an interview. By the end of this course, your GitHub will show **16 completed DSA labs**, which is real proof of your skills.

1. Go to **github.com** and sign up with a professional username (e.g., **ali-raza-dev**, not **cool_boy_786**).
2. Click **New repository**, name it **DSA-Labs**, choose **Public**, tick **Add a README file**, and click **Create**.
3. Inside the repository, click **Add file** → **Create new file** and type **Lab01/README.md** to create a folder for this lab.
4. After completing the lab, upload your notebook: **Add file** → **Upload files**, then commit.
5. In Colab you can also use **File** → **Save a copy in GitHub**.

3.3 LeetCode: Practise Like a Professional

1. Sign up at **leetcode.com**.
2. Open **Problems**, filter by **Easy**, and solve the problems listed in the LeetCode section of this manual.
3. Aim for **at least 2 problems per week**. By Week 16 you should have 30+ solved problems.

4. Part B: What Are Data Structures and Algorithms?

4.1 Data Structure

A **data structure** is a way of **storing and organising data** in a computer so that we can use it efficiently.



Real-life analogy: a general store

Think of a well-run general store. Rice, sugar and flour are kept on one shelf, soaps on another, and cold drinks in the fridge. When a customer asks for something, the shopkeeper finds it in seconds. If everything were thrown into one big pile, finding a packet of salt would take forever.

The items are the data. The way they are arranged is the data structure.
The right arrangement makes work fast.

Data structures are commonly divided into these groups:

Type	Meaning	Examples
Primitive	Basic building blocks provided by the language; store a single value	int, float, char, bool
Non-primitive (Linear)	Elements are arranged one after another in a sequence	Array, Linked List, Stack, Queue

Type	Meaning	Examples
Non-primitive (Non-linear)	Elements are arranged in a hierarchy or network	Tree, Heap, Graph
Hash-based	Elements are found directly using a key	Hash Table (Python dict, set)

4.2 Algorithm

An **algorithm** is a **finite sequence of clear steps** that solves a problem. It is the logic, not the code. The same algorithm can be written in Python, C++ or Java.

Real-life analogy: a recipe

A recipe for making tea is an algorithm: boil water, add tea leaves, add milk and sugar, boil again, pour into cup. The steps are clear, ordered and they finish.

Five properties every algorithm must have

Property	Meaning	Tea example
Input	Zero or more values given from outside	Water, milk, tea leaves, sugar
Output	At least one result is produced	A cup of tea
Definiteness	Every step is clear, with only one meaning	“Boil for 2 minutes”, not “boil for a while”
Finiteness	It must stop after a limited number of steps	The process ends when tea is poured
Correctness	It gives the right output for every valid input	Always produces tea, not coffee

4.3 Data Structure + Algorithm = Program

Niklaus Wirth, a famous computer scientist, wrote a book titled **Algorithms + Data Structures = Programs**. Every software you use is a combination of the two. Choosing a good data structure often makes the algorithm simple and fast.

4.4 Worked Example: Find the Highest Marks

Problem: A teacher has the marks of students: 67, 89, 45, 92, 78. Find the highest marks.

Step 1: Pseudocode (language-independent)

ALGORITHM FindMax(marks)

INPUT : a list of numbers called marks

OUTPUT: the largest number

1. max_value ← marks[0]
2. FOR each value in marks DO
3. IF value > max_value THEN
4. max_value ← value
5. RETURN max_value

Step 2: Dry run (trace by hand)

Step	value	value > max_value ?	max_value
Start	-	-	67
1	67	67 > 67 → No	67
2	89	89 > 67 → Yes	89
3	45	45 > 89 → No	89
4	92	92 > 89 → Yes	92
5	78	78 > 92 → No	92

Step 3: Python code

```
def find_max(marks):
    max_value = marks[0]      # step 1: assume first is largest
    for value in marks:      # step 2: check every value
        if value > max_value: # step 3: found a bigger one?
            max_value = value # step 4: remember it
    return max_value         # step 5: answer

print(find_max([67, 89, 45, 92, 78])) # Output: 92
```

The notebook also prints the dry run automatically, so you can compare it with your hand trace:

Automatic dry run

```
def find_max_trace(marks):
    max_value = marks[0]
    print(f"Start: max_value = {max_value}")
    for step, value in enumerate(marks, start=1):
        bigger = value > max_value
        if bigger:
            max_value = value
        ans = "Yes" if bigger else "No"
        print(f"Step {step}: value={value:>3} | bigger? {ans:3} |
max={max_value}")
    return max_value

find_max_trace([67, 89, 45, 92, 78])
```

 Check the five properties

Input: the list of marks. Output: 92. Definite: each line has one meaning.
Finite: the loop ends after the last item. Correct: it works for any non-empty list.

Think: what happens if the list is empty? (You will fix this in Task 3.)



5. Part C: Python Essentials for DSA

This part covers only the Python you need for DSA. Run every example in your notebook and change the values to see what happens. **Learning to code is like learning to swim: you must get into the water.**

5.1 Variables and Data Types

A **variable** is a name that refers to a value stored in memory. Python decides the type automatically.

```
name = "Ayesha"      # str   (text)
age = 19             # int   (whole number)
cgpa = 3.45          # float (decimal number)
is_enrolled = True   # bool  (True / False)

print(type(name), type(age), type(cgpa), type(is_enrolled))
```

5.2 Input and Output

```
student = input("Enter your name: ")
marks = int(input("Enter your marks: ")) # input() always gives text,
                                         # so convert with int()
print(f"{student} scored {marks} marks.") # f-string formatting
```

Operator	Meaning	Example → Result
+ - * /	Add, subtract, multiply, divide	7 / 2 → 3.5
//	Integer (floor) division	7 // 2 → 3
%	Remainder (modulus)	7 % 2 → 1 (used to check even/odd)
**	Power	2 ** 10 → 1024
== != < > <= >=	Comparison	5 > 3 → True
and or not	Logical	True and False → False

Operators in action

```
print(7 / 2, 7 // 2, 7 % 2, 2 ** 10)
n = 14
print(n, "is", "even" if n % 2 == 0 else "odd")
```

5.3 Conditions: if / elif / else

```
marks = 76
if marks >= 85:
    grade = "A"
elif marks >= 70:
    grade = "B"
elif marks >= 50:
    grade = "C"
else:
    grade = "F"
print("Grade:", grade)          # Grade: B
```

Indentation matters

Python uses **4 spaces** of indentation to group code. In C++ we use curly braces { }. Wrong indentation causes an **IndentationError** or wrong logic.

5.4 Loops: for and while

```
# for loop: repeat for each item
for i in range(1, 6):          # 1, 2, 3, 4, 5 (6 is excluded)
    print(i, "squared =", i * i)

# while loop: repeat while a condition is True
balance = 1000
months = 0
while balance < 2000:
    balance = balance * 1.10    # 10% profit per month
    months += 1
print("Months needed:", months) # 8
```

5.5 Functions: Write Once, Use Many Times

```
def calculate_bill(price, quantity, discount=0):
    """Return total bill after discount (in percent)."""
    total = price * quantity
    return total - (total * discount / 100)

print(calculate_bill(250, 4))      # 1000.0
print(calculate_bill(250, 4, 10))  # 900.0
```

Functions are the heart of DSA. Every operation we will build (insert, delete, search, sort) will be a function.

5.6 Python's Built-in Collections

Python gives us four ready-made collections. Each one is actually a data structure, and in later labs you will learn **how they work inside** and build your own.

Collection	Syntax	Ordered ?	Changeable?	Duplicates?	Studied in
List	[1, 2, 3]	Yes	Yes	Yes	Lab 03 (Arrays)
Tuple	(1, 2, 3)	Yes	No	Yes	Lab 03
Dictionary	{"a": 1}	Yes*	Yes	Keys: No	Lab 12 (Hashing)
Set	{1, 2, 3}	No	Yes	No	Lab 12 (Hashing)

* Since Python 3.7, dictionaries remember insertion order.

List: the most-used collection

```
marks = [67, 89, 45, 92, 78]
print(marks[0])      # 67   first element (index starts at 0)
print(marks[-1])     # 78   last element
print(marks[1:4])    # [89, 45, 92]   slicing
marks.append(55)      # add at end
marks.insert(0, 100)  # add at index 0
marks.remove(45)      # remove first 45
print(len(marks), max(marks), sum(marks))
```

Tuple: fixed data

```
point = (31.5204, 74.3587)    # (latitude, longitude): a fixed pair
print(point[0])
try:
    point[0] = 0               # tuples cannot be changed
except TypeError as e:
    print("Error:", e)
```

Dictionary: key → value pairs

```
prices = {"sugar": 160, "rice": 320, "milk": 220}
print(prices["rice"])        # 320   (very fast lookup by key)
prices["tea"] = 1450         # add new item
for item, price in prices.items():
    print(item, "→", price)
print(prices.get("honey", "not available"))  # safe lookup
```

Set: unique items only

```
roll_numbers = [12, 15, 12, 18, 15, 20]
unique = set(roll_numbers)
print(unique)           # {12, 15, 18, 20}
print(15 in unique)     # True (very fast membership check)
```

List comprehension: a short way to build lists

```
squares = [x * x for x in range(1, 6)]           # [1, 4, 9, 16, 25]
passed = [m for m in [67, 45, 92, 30] if m >= 50] # [67, 92]
print(squares, passed)
```

5.7 Strings

```
word = "Okara"
print(word.upper(), word.lower(), len(word)) # OKARA okara 5
print(word[::-1])                          # arakO (reverse)
for ch in word:
    print(ch, end=" ")                      # O k a r a
```

5.8 How Python Stores Data in Memory (Very Important for DSA)

In Python, a variable does **not** contain the value directly. It is a **label (reference)** pointing to an object in memory. Understanding this now will make **linked lists**, **trees** and **graphs** much easier later, because those structures are built entirely from references.

```
a = [1, 2, 3]
b = a           # b points to the SAME list, not a copy
b.append(4)
print(a)        # [1, 2, 3, 4] a also changed!
print(a is b)   # True: same object in memory

c = a.copy()    # a real copy
c.append(5)
print(a)        # [1, 2, 3, 4] a is safe
```

Mutable (can change in place)	Immutable (cannot change)
list, dict, set	int, float, str, tuple, bool

Connection to future labs

In Lab 08, each node of a linked list will hold a value and a **reference** to the next node, exactly like `b = a` above. If you understand this example, you already understand the core idea of a linked list.

6. C++ Corner

For instructors and students working in C++, here is the same Find-Max algorithm. Notice that the **algorithm (logic) is identical**; only the syntax changes.

C++

```
#include <iostream>
#include <vector>
using namespace std;

int findMax(const vector<int>& marks) {
    int maxValue = marks[0];
    for (int value : marks) {
        if (value > maxValue)
            maxValue = value;
    }
    return maxValue;
}

int main() {
    vector<int> marks = {67, 89, 45, 92, 78};
    cout << "Highest marks: " << findMax(marks) << endl;    // 92
    return 0;
}
```

Concept	Python	C++
Variable type	Decided automatically	Must be declared: int, float, string
Code blocks	Indentation	Curly braces { }
Dynamic list	list	vector<int>
Key-value store	dict	unordered_map<string,int>
Unique items	set	unordered_set<int>
Print	print(x)	cout << x << endl;

7. Common Mistakes Beginners Make

Mistake	Example	Fix
Forgetting to convert input	<code>age = input()</code> then <code>age + 1</code> → error	<code>age = int(input())</code>
Index starts at 1 (wrong)	<code>marks[5]</code> on a 5-item list → <code>IndexError</code>	Valid indexes are 0 to <code>len-1</code>
Using <code>=</code> instead of <code>==</code>	<code>if x = 5:</code>	<code>if x == 5:</code>
Wrong indentation	Mixing tabs and spaces	Always use 4 spaces
Copying a list with <code>=</code>	<code>b = a</code> changes both	<code>b = a.copy()</code>
Not testing edge cases	Code fails on an empty list	Test empty, single-item and large inputs

8. Lab Tasks

Solve all tasks in `Lab01_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Student Result Card

Write `result_card(name, marks)` that takes a name and a list of 5 subject marks (each out of 100), prints the total, percentage and grade ($A \geq 85$, $B \geq 70$, $C \geq 50$, otherwise F), and **returns** (`total, percentage, grade`).

```
Input : result_card("Hamza", [78, 85, 90, 66, 72])
Output: Hamza | Total: 391/500 | Percentage: 78.2% | Grade: B
```

Task 2 Even or Odd Counter

Given a list of integers, return (`even_count, odd_count`) using the `%` operator.

```
Input : [12, 7, 9, 20, 33, 18]
Output: (3, 3)
```

Task 3 Safe Find-Max

Improve `find_max` so that it returns `None` for an empty list instead of crashing. **Do not** use Python's built-in `max()`.

Task 4 General Store Bill

A store keeps prices in a dictionary. Write `make_bill(prices, cart)` where `cart` is a dictionary of item $\rightarrow$ quantity. Print each line and the totals. If the subtotal is above Rs. 5,000, apply a 5% discount. If an item is not in the price list, print a warning and skip it. **Return** the final total.

```
prices = {"sugar": 160, "rice": 320, "oil": 560, "tea": 1450}
cart   = {"rice": 5, "oil": 3, "tea": 2, "honey": 1}
rice   x 5 = 1600
oil    x 3 = 1680
tea    x 2 = 2900
Warning: 'honey' not found
Subtotal: 6180 | Discount: 309.0 | Total: 5871.0
```

Task 5 Palindrome Checker

A palindrome reads the same forward and backward (e.g., “madam”, “Never odd or even”). Ignore spaces and capital letters.

Solve it **twice**: once using slicing `[::-1]`, and once with a loop using two indexes (one from the start, one from the end). The second method is the **two-pointer technique**, which you will use again in Lab 04.

Task 6 🟡 Character Frequency

Write `char_frequency(text)` that returns a dictionary showing how many times each character appears (ignore spaces). Then print the most frequent character.

```
Input : "data structures"
Output: {'d': 1, 'a': 2, 't': 3, 's': 2, 'r': 2, 'u': 2, 'c': 1, 'e': 1}
Most frequent: 't' (3 times)
```

Task 7 🔴 Second Highest Marks

Find the **second highest unique** value in a list in **one pass** (a single loop), without sorting and without `max()`. Return `None` if it does not exist.

```
[67, 89, 45, 92, 78] → 89
[50, 50, 50]         → None
[10, 20, 20]         → 10
```

Task 8 🔴 Two Sum (Your First Interview Problem)

Given a list of numbers and a target, return the **indexes** (`i, j`) of two numbers that add up to the target (or `None`). First solve it with two nested loops. Then solve it using a **dictionary** in a single loop. Run the bonus experiment in the notebook to see the speed difference. You have just discovered why data structures matter, which is the topic of Lab 02!

```
two_sum([2, 7, 11, 15], 9) → (0, 1)    because 2 + 7 = 9
```

🏆 9. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub `DSA-Labs/Lab01` folder.

#	Problem	Level	Concept practised
1480	Running Sum of 1d Array	Easy	Loops, lists
1672	Richest Customer Wealth	Easy	Nested lists, sum
412	Fizz Buzz	Easy	Conditions, %
9	Palindrome Number	Easy	Two-pointer / reversal

#	Problem	Level	Concept practised
1929	Concatenation of Array	Easy	List operations
1	Two Sum	Easy	Dictionary (hash map), same as Task 8



10. Home Task

1. Complete any lab tasks you could not finish in the lab.
2. Create your **DSA-Labs** repository on GitHub and upload your completed **Lab01_Student.ipynb** in the **Lab01** folder.
3. Write a short **README.md** in the Lab01 folder listing what you learned in 3–5 bullet points.
4. **Mini-project (choose one):** (a) a **Contact Book** using a dictionary with add, search, update, delete and display options in a menu loop; or (b) a **Quiz Game** that stores questions and answers in a list of dictionaries and shows the final score.
5. Solve at least 2 of the LeetCode problems listed above.

? 11. Viva Questions

1. What is a data structure? Give one real-life example.
2. What is an algorithm? How is it different from a program?
3. Name the five properties of an algorithm and explain finiteness.
4. What is the difference between primitive and non-primitive data structures?
5. Differentiate between linear and non-linear data structures with examples.
6. What is the difference between a list and a tuple in Python?
7. Why is searching in a dictionary or set faster than in a list? (Hint: Lab 12)
8. What happens when you write **b = a** for a list? How do you make a real copy?
9. Which data structure is used for the Undo feature? Why?
10. Why is DSA still important when AI tools can write code?



12. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



13. References and Further Learning

- Official Python Tutorial: docs.python.org/3/tutorial
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: pythontutor.com; animated data structures: visualgo.net.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.